COM3021 – Data Science at Scale

Batch processing

DR HUGO BARBOSA

Introduction

- There are three classes of systems

  Services (online systems)

  - A service waits for a request and handles it as quickly as possible and sends a response back.

    The primary performance metrics are response time and availability

  Batch processing systems (offline systems)

  - A batch processing system takes a large amount of input data, runs a *job* to process it, and produces some output data.

    Jobs can take from minutes to days

    Batch jobs are often scheduled to run periodically.

    - The primary performance measure of a batch job is usually *throughput*

  Stream processing systems (near-real-time systems)

  Stream processing is somewhere between online and offline/batch processing

  A stream processor consumes inputs and produces outputs

  - Operates on events shortly after they happen

Introduction

- Reading a log file and counting occurrences
- Unix

```
cat /var/log/nginx/access.log | ❶
  awk '{print $7}' | ❷
  sort             | ❸
  uniq -c          | ❹
  sort -r -n       | ❺
  head -n 5          ❻
```

Introduction

- The Unix philosophy
- Ruby

```ruby
counts = Hash.new(0) ❶

File.open('/var/log/nginx/access.log') do |file|
  file.each do |line|
    url = line.split[6] ❷
    counts[url] += 1 ❸
  end
end

top5 = counts.map{|url, count| [count, url] }.sort.reverse[0...5] ❹
top5.each{|count, url| puts "#{count} #{url}" } ❺
```

The Unix Philosophy

- Make each program do one thing well. To do a new job, build afresh rather than complicate old programs by adding new “features”.
- Expect the output of every program to become the input to another, as yet unknown, program. Don’t clutter output with extraneous information. Avoid stringently columnar or binary input formats. Don’t insist on interactive input.
- Design and build software, even operating systems, to be tried early, ideally within weeks. Don’t hesitate to throw away the clumsy parts and rebuild them.
- Use tools in preference to unskilled help to lighten a programming task, even if you have to detour to build the tools and expect to throw some of them out after you’ve finished using them.

MapReduce

- A batch processing algorithm
- Low-level programming model
- A single MapReduce job is comparable to a single Unix process: it takes one or more inputs and produces one or more outputs.
- Distributed filesystems:

  Colossus (successor of Google File System GFS)

  HDFS (Hadoop Distributed File System)

  GlusterFS

  QFS (Quantcast File System)

MapReduce

- Job execution

  Read a set of input files, and break it up into records.

  Call the mapper function to extract a key and value from each input record.

  Sort all of the key-value pairs by key.

  Call the reducer function to iterate over the sorted key-value pairs.

MapReduce

- *Mapper*

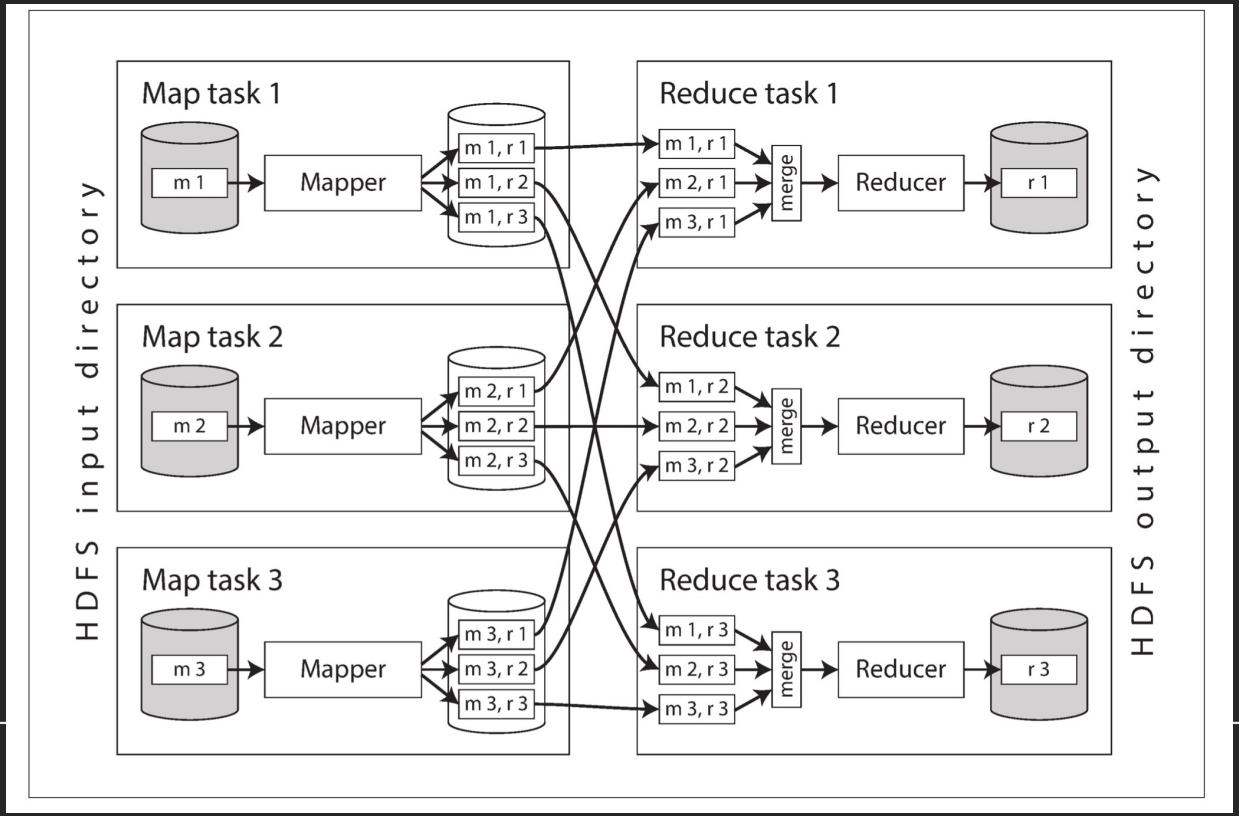
  The mapper is called once for every input record, and its job is to extract the key and value from the input record. For each input, it may generate any number of key-value pairs (including none). It does not keep any state from one input record to the next, so each record is handled independently.

- *Reducer*

  The MapReduce framework takes the key-value pairs produced by the mappers, collects all the values belonging to the same key, and calls the reducer with an iterator over that collection of values. The reducer can produce output records (such as the number of occurrences of the same URL).

Distributed execution of MapReduce

- The key advantage of MapReduce is parallelism



MapReduce workflow

- The range of problems you can solve with a single MapReduce job is limited.
- MapReduce jobs are normally chained together into *workflows*

  The output of one job becomes the input to the next job.
- The Hadoop Map- Reduce framework does not have any particular support for workflows, so this chaining is done implicitly by directory name:

  the first job must be configured to write its output to a designated directory in HDFS, and the second job must be configured to read that same directory name as its input.

Reduce-Side Joins and Grouping

- When we talk about joins in the context of batch processing, we mean resolving all occurrences of some association within a dataset.

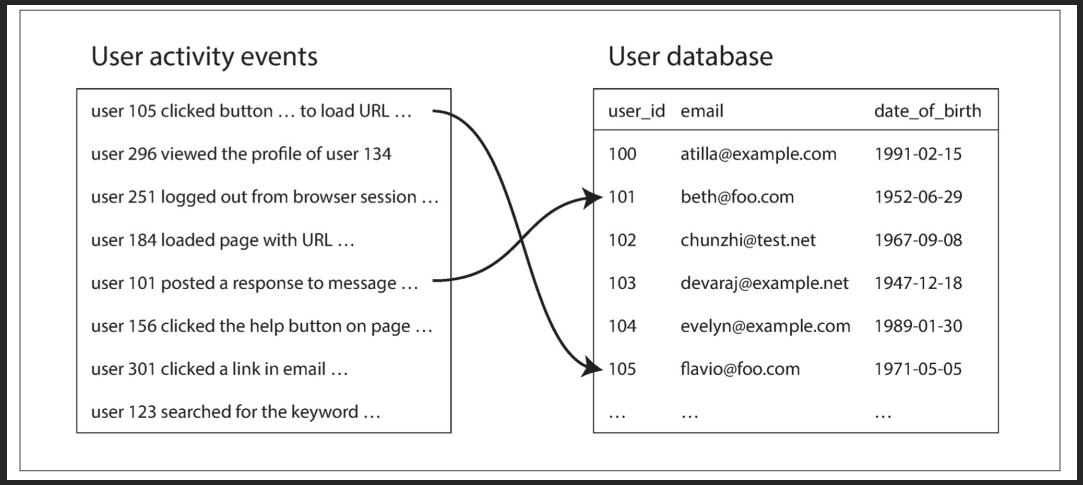
  The job is processing the data for all users simultaneously

Reduce-Side Joins and Grouping

- Ex: Joining activities and users data

  The simplest implementation of this join would go over the activity events one by one and query the user database for every user ID it encounters.

  In order to achieve good throughput in a batch process, the computation must be (as much as possible) local to one machine.



Reduce-Side Joins and Grouping

- One set of mappers would go over the activity events (extracting the user ID as the key and the activity event as the value)
- Another set of mappers would go over the user database (extracting the user ID as the key and the user's date of birth as the value).